WASP Software Engineering Course Assignment

Nils Dunlop, 20010127-2359, Chalmers University, nilsdu@chalmers.se

September 1, 2026

1 Introduction

My research area is part of TIMED (Time-Resolved Imaging and Multi-Channel Evaluation of Cellular Dynamics), a WASP/DDLS NEST project run jointly by Chalmers, Uppsala University and KTH. The project aims to establish a framework for collecting, processing and analyzing time-resolved cellular imaging data. The resulting models are to be finally validated on patient-derived ovarian cancer cells.

The experimental partners operate automated live-cell imaging platforms: cells are treated with a compound at a set concentration and imaged every hour for 48 hours across three fluorescence channels. The current dataset comprises 10 plates and 10 cell lines, totaling roughly 750,000 images and 6.15 TB. I work primarily on the predictive and generative models, but I am also one of the main developers of the shared processing pipeline. Two concrete tasks we are pursuing are predicting whether cells are alive or dead at 48 hours given only the first 24 hours of a trajectory, and predicting an optimal concentration for a given cell line.

The project is meant to build into a closed loop rather than a single model, where models propose promising candidates, the experimental partners test them and the resulting images are fed back into training. This means the model is only one component of a larger system: images are acquired at the partner institutions, while training runs on shared compute clusters, so moving, versioning and validating data are necessary steps before any modeling can begin.

The project is near the end of its first year, and much of that time was spent generating the dataset so the pipeline is only now being developed. Three problems shape how we approach developing it. First, we expect many people to use the system for different research questions, so the architecture must be modular enough that changes in one part do not propagate technical debt through the rest. Second, we treat our experimental labels as ground truth, but they are fuzzy in reality. Therefore, we plan to perform RNA extraction on a small subset to obtain stronger labels for validation, though the cost prevents doing this at scale. Third, images arrive in batches from both KTH and Uppsala over the course of the project, and differences in experimental setup between the sites and over time mean the data a model was trained on will not match what it later encounters. We therefore need a metadata specification that records the experimental conditions for each batch, both so that these differences can be tracked and so that the metadata itself can be fed to the model to help it account for batch effects. The following sections relate these three problems to the course material.

2 Lecture Principles

2.1 Requirements Engineering

Especially today, with AI coding assistants shortening development time, truly understanding what your stakeholders want has become more important. I see this clearly in my own project. I could develop many different models, for example generative models of cell images that predict intermediate states. For the biologists I work with, however, such models are of limited value: they are difficult to trust, since there is no established way to verify that a generated image is biologically plausible and they do not directly help determine the effect of a drug.

Predictive models that can be validated against experimental outcomes and that simplify the biologists existing workflow are far more useful. What makes a model valuable here is therefore not its accuracy alone but whether its output can be checked and acted upon. Developing requirements

together with the biologists, rather than deciding on my own what is worth building has been essential to ensuring that the models create value. Furthermore, interdisciplinary work makes exchange run in both directions: the biologists help me interpret my results, while I contribute to decisions on how the data should be stored and handled.

2.2 Technical Debt

When developing a longer-term code solution, the focus needs to be on establishing strong architectural choices, design patterns, coding practices and modularity. Otherwise, technical debt and antipatterns accumulate and the system becomes harder to fix or build upon. Technical debt can be mitigated but rarely removed entirely.

In AI-based systems this debt is not only in the code. A preprocessing choice made once during segmentation propagates into every profile and every model trained afterwards, and unlike a code defect it is not easy to detect. The components are entangled so a change in one place has effects that are hard to trace.

In our project, we plan to use a modular architecture with a fixed base structure of segmentation, feature extraction, profile creation and model training. If a researcher wants to change a component, this can be done through a configurable YAML file together with minor code changes to handle dependencies. We expect fellow researchers to ask different questions and to need slightly different setups and this lets them do so while still working within the same repository. The trade-off is that each configurable boundary is a commitment to keep that interface stable and every new option adds a combination that must eventually be tested.

3 Guest Lecture Principles

3.1 Semi-Executable Stack

Per Lenberg's semi-executable stack describes software engineering as a set of layers, from code up through prompts, workflows, controls, operating logic and societal fit, and shows how each layer changes when the work becomes AI-mediated. The layer that matters most for my work is controls, where reviews, approvals and QA gates are replaced by automated evaluation and confidence thresholds.

In a recent project on interpretability in cell image models, I used Gemini as an LLM judge to check that the top SHAP features reported by a model were in fact visible in the corresponding images. Only once the model was picking up useful biology did we bring in expert biologists for further analysis. The automated check allowed quick iteration on the model and the feature engineering, which was necessary because expert validation is expensive, and it let us prioritize where manual verification was actually needed.

This made the automated check a component of the system rather than a convenience, so it needed validating in its own right. We compared the LLM judgments against the biologists assessments and found them relatively aligned. However, for more difficult cases, such as lower concentrations or complex drugs where the model was less confident in its features, even the biologists could not reliably confirm the feature from the image alone. Where the signal in the image was clear the LLM judgments were strong, but where the images were fuzzier the experts could still find nuances the judge missed. The gate is therefore reliable within a range, and knowing where that range ends is what makes it usable as a threshold rather than a replacement for expert review.

3.2 Hidden gaps become visible

The guest lecturer made the point that as AI begins to replace human interpretation and informal coordination, unclear responsibilities and hidden gaps become visible. I recognized this in my own research team. Over the past year we repeatedly ran into the same administrative difficulties when writing papers, such as polishing figures, handling paper templates and agreeing on how we write. Together with our supervisor we set up a team repository with tutorials for common tasks, along with a few skills for the processes we perform frequently.

What made this an instance of Lenberg's point was what building it required. The standard we were working towards was our supervisor's, and it existed mainly as her judgment on individual drafts rather than as anything written down. Automating the work meant first converging on what

that standard actually was, which took several iterations of feedback before we could express it as instructions an agent could follow. The result kept the quality she expected, but it only became reusable once the implicit criteria were made explicit.

4 Data scientists, software engineers, and AI engineers

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction?

The chapters describe data scientists as preferring to build models and algorithms, spending much of their time gathering and cleaning data, while software engineers focus on delivering products that meet user or stakeholder needs [1]. Data scientists work in an exploratory workflow built around notebooks and evaluate their work by accuracy on held-out data, whereas software engineers work at scale and take a system-wide view.

I am a software engineer turned data scientist and have noticed these differences clearly, particularly in how tasks are prioritized. Software engineers tend to consider several architectural options before implementing anything, with the end user in mind. Data scientists prototype quickly and run training in notebooks, which makes it hard to build a durable code base. I think the chapters give a reasonable account of the stereotypes. However, the "unicorns" they describe do exist, and in my experience which side someone leans towards comes down largely to educational background rather than to any inherent division of the work.

The chapters argue that production ML requires a system-wide view. How does this apply to your own research area?

Research does tend to be model-focused, though usually with the wider system somewhere in mind. In TIMED the system-wide view is unavoidable. The three problems described in the introduction, modularity across many users, labels that are only proxies, and data arriving from different sites over time, are all system properties rather than model properties. Partly because of the interdisciplinary team and partly because of my background, I have taken on an additional system developer role in the project.

Will data scientists and software engineers remain distinct roles?

Many data science projects are becoming more applied, so data scientists increasingly need software architecture and web development skills. Equally, most software now has some AI component, so software engineers will pick up data science skills. I think the crossover is driven by the task rather than by the role. An ML researcher may never need software engineering skills, and a systems architect may never need data science ones.

Where does AI engineering fit in this picture?

I see AI engineering as data science with a focus on delivery and users rather than on the model itself. In many applications the model is not built from scratch but obtained from a provider and connected to other services, such as MCP servers or cloud infrastructure. This does not hold everywhere, since domains like mine still require training models on data that no provider has, but the assembly-and-integration pattern is increasingly the common case.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

These developments make the distinctions considerably fuzzier. With a data science background and an AI assistant, you can get most of the way to building the web interface for a model or adding an AI component to a platform. The result is fewer clearly separated roles and more general AI software developers. What this does not remove is the judgment about what should be built, which is where the remaining difference between the roles seems to sit.

5 Paper analysis

5.1 Bringing Machine Learning Models Beyond the Experimental Stage with Explainable AI

Core ideas and their SE importance

The paper describes how although AI-enabled technologies are emerging across many sectors, few companies manage to move models beyond the experimental phase and into production systems [2]. It follows a case study at Jyske Bank, where a previously unused ML model was integrated into a mission critical IT system for bond pricing. The authors attribute models staying experimental to the different development cultures in data science and software engineering, a lack of established architectures and processes supporting integration, and a lack of trust from expert users. Their contribution is three guidelines extending Zinkevich's best practices for ML engineering [3], all built around explainable AI: using XAI to gather end-user feedback, using global SHAP values for model validation so that domain experts can verify the most important features match their intuition, and monitoring the explanations rather than only the predictions in order to detect behavioral drift and training/serving skew.

Making this work in production was itself an engineering problem, since auctions last at most 30 seconds. Computing full SHAP values would have taken around 20 minutes, so the authors measured consistency and accuracy against sample size and settled on 700 samples, while anchors were pre-computed because 100 of them covered 90% of auctions. The evaluation was honest about the limits: dealers looked at the predicted price but rarely at the explanations, and found them convincing only sometimes. The authors nonetheless argue that explanations were necessary for getting the model out of the experimental stage, and their one-year follow-up shows that once trades were priced automatically, explainability mattered less while monitoring mattered more.

Relation to my research

The paper relates closely to AI for biology, where models face the same difficulty in reaching production use, largely because expert biologists cannot trust the model's predictions. A recent project of mine investigated whether the top features of cell image models are actually present in the images themselves to make such models interpretable enough to be validated. We used a two-step evaluation: Gemini first acted as a judge on whether the SHAP features corresponded to what was visible in the image, which allowed us to iterate quickly on model training and feature engineering. Only once the model appeared both accurate and interpretable did we bring in expert biologists for manual validation.

This is the second guideline of Mikkelsen et al. [2], which proposes computing global SHAP values so that domain experts can verify that the features the model considers most important are also the most relevant. The difference is that we inserted an automated judge before the experts, because expert time is the scarce resource in our project. Their third guideline is also directly applicable, since they argue that XAI is especially useful for monitoring models where performance cannot be monitored live or where true labels cannot be generated automatically. That describes our situation, where ground truth is fuzzy and RNA extraction is costly to run at scale.

The settings differ however in a notable way. Their end users are five dealers reading tabular features under a 30-second deadline, so their constraint is speed. Ours are biologists judging whether an effect is visible in an image, and for more complex cases even they could not reliably confirm the feature. Our constraint is therefore the weakness of the oracle rather than the latency of the explanation.

Integration into a larger AI-intensive project

A suitable larger project is the closed-loop TIMED platform described in the introduction, where models propose promising compounds and concentrations, the experimental partners test them and the resulting images are fed back into training. Several elements of the paper transfer directly. Their ML dashboard allows a person with a business rather than a data science background to compare a retrained model against the current one using metrics defined together with domain experts and to deploy it. In TIMED the equivalent would let a biologist compare models on biologically meaningful metrics rather than on loss values, which matters because no single person owns the pipeline. Their practice of tracking global SHAP values over time to detect drift in feature importance would translate to tracking which morphological features drive predictions

across plates, cell lines and sites, giving an early signal of batch effects. Their guardrails, which route trades to a human when needed, correspond to routing low-confidence predictions to the biologists rather than into the candidate list.

My own research supplies the predictive component of this loop and the interpretability work supplies the mechanism by which its outputs become trustworthy enough to act on.

Adaptation of my research

Two changes follow from the paper. First, I would treat explanation stability as a release criterion rather than an analysis afterthought, so that a significant change in feature importance between model versions has to be investigated before deployment. Second, I would group correlated features when computing SHAP values rather than attributing importance to each in isolation. The authors report the same criticism from their own users, that correlated features were difficult to separate in explanations, and suggest coalitions of Shapley values as a possible remedy while noting that no systematic study exists. This is more pressing for image data, where morphological features are rarely independent and grouped attributions would give biologists explanations at the level of biological structures rather than individual pixels or descriptors.

5.2 A Meta-Summary of Challenges in Building Products with ML Components

Core ideas and their SE importance

The second paper argues that building an ML-enabled product requires a shift from a model-centric view to a pipeline- and system-centric one [4]. The work is a meta-summary of 50 empirical studies, together covering 947 interview participants and 3811 survey respondents. The authors extracted 520 reported challenges and grouped them into requirements engineering, architecture, design and implementation, model development, data engineering, quality assurance and organization and teams. Rather than proposing a technique, the paper is useful as an evidence-based risk checklist of common pitfalls when ML enters a software project.

Several findings are especially relevant to AI engineering. First, 11 of 50 studies report difficulty moving from exploratory model code to an automated production-quality pipeline. A further 11 report implicit data and tooling dependencies with insufficient modularity. This produces "pipeline jungles" and what practitioners describe as "change anything changes everything" behavior. Second, data quality is reported as problematic in 17 studies. Training/serving skew and data versioning are each raised in 7. Practitioners describe being unable to answer which data a model was trained on. Finally, conventional testing is insufficient. Practitioners struggle to define what mistakes are acceptable and to obtain representative labeled test data. They also struggle to test the interactions among data, models, pipelines, infrastructure and the complete system.

Relation to my research

The paper documents the three problems we anticipate in TIMED.

The first is building a production pipeline after the fact. Without deliberate structuring from the start, implicit data and tool dependencies accumulate and "pipeline jungles" emerge. This is the risk in a shared repository where interfaces are set by convention rather than by contract.

The second is defining acceptable model errors and obtaining representative labeled test data. Our viability labels are proxies and the measurement that would resolve them is too costly to run at scale.

The third is training/serving skew and data versioning. Images arrive from two sites over several years, so what a model was trained on and what it later encounters will differ and will need to be handled with care to mitigate batch effects.

Integration into a larger AI-intensive project

Within the closed-loop TIMED platform described above, the paper serves as a design-time checklist. It summarizes what went wrong for practitioners across 50 studies, which lets us treat those failure modes as things to design against while the system is still being built. This is especially useful since we are currently within the design process and not maintaining a finished one.

Concretely, three of its findings would become design requirements. Components should communicate through versioned interfaces rather than shared assumptions, so that a change in segmentation does not silently propagate into every downstream model. A batch should carry the metadata needed to identify it before it enters training. Evaluation should test the pipeline end to end rather than the model alone, since the paper reports that system-level testing is the part most often neglected. My own research supplies the predictive component of the loop, and the pipeline work supplies the surrounding structure that makes it usable by others.

Adaptation of my research

We already split data by plate and use leave-one-compound-out validation, but we do not test transfer across sites or forward in time. Adding leave-site-out and time-ordered splits would test whether a model trained on earlier batches can generalize to data collected later at another institution.

I would also improve traceability. While Weights & Biases records configurations and metrics, it does not specify which data each model saw. Linking every run to an immutable batch manifest containing image IDs, acquisition metadata and preprocessing versions would make this information easy to retrieve.

Finally, I would involve the whole team in reviewing the pipeline architecture during our weekly meetings. Since most collaborators will use rather than develop the pipeline, early input could reduce the need for major architectural changes later.

The paper offers no concrete solutions, but documents common practical challenges. Adapting my research therefore means addressing the most relevant of these challenges early.

6 Evidence, originality, and GenAI-use transparency

The two papers analyzed in Section 5 are peer-reviewed full papers from the IEEE/ACM CAIN conference [2, 4]. Section 4 draws on Kästner [1], an openly available book based on a taught course. Zinkevich's best practices [3] are practitioner documentation rather than peer-reviewed research and are cited second-hand via Mikkelsen et al.

I place the most trust in the CAIN papers because they are peer-reviewed, provide sufficient methodological detail, and acknowledge their limitations. I treat the CMU book as an informed synthesis rather than primary evidence, particularly since its author also co-authored Nahar et al. Zinkevich's guide I treat as practitioner experience from one organization and give it correspondingly little weight.

I used generative AI, specifically Claude Sonnet 5 and Claude Opus 5, to help identify candidate papers and to polish and structure text I had drafted myself. I read and selected the papers myself. The analysis and all connections to my own research are my own.

References

[1] C. Kästner, *Machine Learning in Production: From Models to Products*. MIT Press, 2025. [Online]. Available: https://mlip-cmu.github.io/book/

[2] N. W. Mikkelsen, L. S. Pedersen, M. Hussain, V. Foged, and E. Kindler, "Bringing machine learning models beyond the experimental stage with explainable AI," in *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2025, pp. 119–129.

[3] M. Zinkevich, "Rules of machine learning: Best practices for ML engineering," Google Developers documentation, 2017. [Online]. Available: https://developers.google.com/machine-learning/guides/rules-of-ml

[4] N. Nahar, H. Zhang, G. Lewis, S. Zhou, and C. Kästner, "A meta-summary of challenges in building products with ML components – collecting experiences from 4758+ practitioners," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*. Los Alamitos, CA, USA: IEEE Computer Society, 2023, pp. 171–183.